Table of contents

Chapter 3. Network-wide ad blocking at home with Pi-hole

> **The payoff of this chapter:** every device on your home network, laptops, phones, the smart TV, game consoles, IoT gadgets, has ads and trackers stripped out *before they ever load*, with zero per-device configuration.

Across Chapters 1–2 you put an always-on Raspberry Pi (`homelab`) on your tailnet and gave it its first job, Audiobookshelf. Pi-hole is the natural second tenant: it's a **DNS sinkhole**, a DNS server that answers "no such host" for domains known to serve ads, trackers, and malware, and forwards everything else to a real upstream resolver. Because it works at the DNS layer, it blocks ads in *every* app and browser at once, including places a browser extension can't reach (smart TVs, mobile apps, in-game ads).

This part is entirely about your **home network**. Pi-hole becomes the DNS server for your house, set once at the router so every device on your Wi-Fi inherits it automatically, no per-device tweaking, no remote access, no VPN to think about. Just clean DNS for everything under your roof.

What you are building (and why this design)

The plan has two halves:

1. **Run Pi-hole on the Pi** (in Docker, alongside Audiobookshelf).
2. **Point your home network at it** by setting Pi-hole as the DNS server in your router, so every device that joins your Wi-Fi automatically uses it.

That router-level step is what makes this effortless: you configure one setting in one place, and every current and future device on your home network is covered without touching any of them individually.

Part A: Run Pi-hole on the Pi

Step 1: Free up port 53 on the Pi

This is the one host-level snag, and it bites almost everyone, so we do it first. Pi-hole needs to listen on **port 53** (the DNS port). But Raspberry Pi OS (like Debian/Ubuntu) ships `systemd-resolved`, which runs a small "stub" DNS listener bound to `127.0.0.53:53`. Two programs can't own the same port, so Pi-hole will fail to start until you evict the stub.

The surgical fix is to disable *only* the stub listener while leaving `systemd-resolved` running to handle the Pi's own name resolution:

```
# 1. Disable the stub listener via a drop-in (survives package upgrades)
sudo mkdir -p /etc/systemd/resolved.conf.d
printf '[Resolve]\nDNSStubListener=no\n' \
  | sudo tee /etc/systemd/resolved.conf.d/no-stub.conf

# 2. Point the Pi's own resolver at resolved's real file, not the dead stub
sudo ln -sf /run/systemd/resolve/resolv.conf /etc/resolv.conf

# 3. Apply the change
sudo systemctl restart systemd-resolved

# 4. Confirm port 53 is now free (this should print nothing)
sudo ss -tulpn | grep ':53'
```

> Why a drop-in file instead of editing the main config
>
> Dropping a file into `/etc/systemd/resolved.conf.d/` rather than editing the main `resolved.conf` keeps your change isolated and reversible: undoing it is `sudo rm /etc/systemd/resolved.conf.d/no-stub.conf` followed by a restart, with no risk of clobbering a setting a future package update wants to change in the main `resolved.conf`.

If step 4 prints a line mentioning `systemd-resolve`, the stub is still bound, re-check that the drop-in saved correctly and that you restarted the service.

Step 2: Give the Pi a fixed LAN IP

Pi-hole's address must never change, because in a moment your whole network will point its DNS at it, and a Pi-hole whose IP drifts is a house-wide DNS outage. So before configuring anything, pin the Pi to one LAN address.

In your **router's admin page**, find the DHCP settings and add a **reservation** (sometimes called a *static lease*): tell the router to always hand the Pi the same IP. Match it by the Pi's MAC address, or pick the Pi out of the list of connected devices. Note the address it gets (for example `192.168.1.50`); if you need the Pi's current IP, run `hostname -I` on the Pi. You'll use this address as `PIHOLE_IP` in the next step.

No reservation option on your router? You can set a static IP on the Pi itself instead, but a router reservation is simpler and the usual way.

Step 3: Run Pi-hole in Docker

You already have a Docker workflow from Chapter 1, so we'll keep Pi-hole in the same world rather than installing it natively. That means one lifecycle to learn (`docker ...`), declarative config you can back up as a file, and no new system packages on the host.

We'll keep this to one clean, self-contained Compose project: a single folder holding one `compose.yaml`, one `.env` for your settings and secrets, and one `.gitignore`. You can read it, back it up, and version-control it without ever exposing a password.

1. Make the folder.

```
mkdir -p ~/pihole
cd ~/pihole
```

2. Create the `.env`. This is the only place your timezone, Pi IP, and admin password live:

```
cat > .env <<'EOF'
TZ=America/Denver
PIHOLE_PASSWORD=replace-this-with-a-strong-password
PIHOLE_IP=192.168.1.50
EOF
chmod 600 .env      # readable only by you
```

Set `TZ` to your timezone and `PIHOLE_IP` to the address you reserved in Step 2. For the password, type a strong one, or generate one and drop it straight in:

```
# optional: replace the placeholder with a random password
sed -i "s|^PIHOLE_PASSWORD=.*|PIHOLE_PASSWORD=$(openssl rand -base64 18)|"
.env
```

> Why `PIHOLE_IP`, and why bind the ports to it
>
> The compose file below publishes Pi-hole's ports on `${PIHOLE_IP}` specifically (e.g. `192.168.1.50:53`) rather than on every interface. Serving on the one LAN address you actually use is tidier and a little safer, Pi-hole listens where it needs to and nowhere else. Make `PIHOLE_IP` the **static or DHCP-reserved** address you reserved in Step 2, so it never changes underneath you.

3. Create `compose.yaml`.

```
cat > compose.yaml <<'EOF'
services:
```

```
  pihole:
    container_name: pihole
    image: pihole/pihole:latest

    ports:
      - "${PIHOLE_IP}:53:53/tcp"
      - "${PIHOLE_IP}:53:53/udp"
      - "${PIHOLE_IP}:80:80/tcp"

    environment:
      TZ: "${TZ}"
      FTLCONF_webserver_api_password: "${PIHOLE_PASSWORD:?set
PIHOLE_PASSWORD in .env}"
      FTLCONF_dns_upstreams: "1.1.1.1;1.0.0.1"
      FTLCONF_dns_listeningMode: "ALL"

    volumes:
      - ./etc-pihole:/etc/pihole

    restart: unless-stopped
EOF
```

There is **no password in this file**, only a `${PIHOLE_PASSWORD}` reference that Compose fills in from `.env` at startup. The `${PIHOLE_PASSWORD:?…}` form makes Compose stop with a clear error if that variable is missing, so you can't accidentally launch with a blank password.

4. Create the `.gitignore` so the secret and the runtime data can never be committed:

```
cat > .gitignore <<'EOF'
.env
etc-pihole/
EOF
```

> **!** The pattern, in one line
>
> **If it's a password, token, or key, it goes in `.env`, and `.env` goes in `.gitignore`.** The `compose.yaml` holds only `${VARIABLE}` references; the real values live in `.env`, which never leaves your Pi. Every later service in this series uses the exact same pattern.

5. Start it.

```
docker compose up -d
docker compose logs --tail 20      # watch it come up
```

Why port 80 now, and why it moves to 8081 later

A web UI conventionally lives on **port 80**, the default HTTP port, which is why Pi-hole's admin sits there for now. We're only borrowing it. In Chapter 4 we add a reverse proxy that has to **own** port 80, because that's the port a browser connects to when you type a bare name like `pihole.home` with no `:port`. Only one program can hold a host port, so the front door (the proxy) gets 80 and Pi-hole's web UI moves to **8081** then. You'll barely notice the number, since you'll reach it as `pihole.home` anyway. Why 8081 and not 8080? Counter-intuitively, **8080 is the most common alternate HTTP port** (dev servers, lots of containers grab it), so it's the one most likely to be taken; 8081 is just one past it. Until then, make sure nothing else holds port 80: Audiobookshelf (Chapter 2) uses 13378 so it's clear, but `sudo ss -tlpn | grep ':80'` confirms it before `docker compose up`.

What the key settings do

- **`FTLCONF_*` environment variables** are Pi-hole v6's configuration mechanism. Each one maps to a key in Pi-hole's single config file (`/etc/pihole/pihole.toml`): `FTLCONF_<section>_<key>` → `[section] key`. Anything you set this way becomes **read-only in the web UI**, it's re-applied from the environment on every container start, which is exactly what you want for reproducible, file-defined config.
- **`FTLCONF_webserver_api_password`** is the v6 replacement for the old `WEBPASSWORD` variable. If you omit it, Pi-hole generates a random password and prints it to the container log on first boot.
- **`FTLCONF_dns_upstreams`** is who Pi-hole asks when a domain *isn't* blocked. Cloudflare (`1.1.1.1`) is fast; Quad9 (`9.9.9.9`) adds its own malware filtering. Pick one.
- **`FTLCONF_dns_listeningMode: "ALL"`** tells Pi-hole to answer queries arriving on any interface. In Docker's bridge network, your LAN clients' queries reach Pi-hole *through* the Docker gateway rather than appearing to come from your local subnet, so the stricter "local only" mode would drop them. `ALL` is the documented working choice for a Dockerized Pi-hole.

Pi-hole v6 is meaningfully different from the v5 you'll find in old guides

Pi-hole **v6** (early 2025) collapsed several moving parts into one. There's no more `lighttpd` web server and no PHP, the `pihole-FTL` process now serves the DNS resolver, the web dashboard, *and* a REST API itself. Configuration moved from a pile of files (`setupVars.conf`, `pihole-FTL.conf`, dnsmasq snippets) to a single `/etc/pihole/pihole.toml`. The commands changed too: `whitelist`/`blacklist` became `pihole allow`/`pihole deny`, and the password command is now `pihole setpassword`. If a tutorial tells you to edit `setupVars.conf`, it predates v6, ignore it.

Step 4: Set the admin password and log in

The web UI lives at `http://<pi-ip>/admin`. On your home network use the Pi's LAN IP (e.g. `http://192.168.1.50/admin`); find it with `hostname -I` on the Pi.

Because the password comes from `PIHOLE_PASSWORD` in your `.env`, it's already configured, log in with whatever value you put there. To change it later, edit `.env` and recreate the container:

```
# edit ~/pihole/.env, then:
cd ~/pihole && docker compose up -d --force-recreate
```

> ⚠ Don't fight your own `.env`
>
> Because the password is pinned by the `FTLCONF_webserver_api_password` environment variable, running `pihole setpassword` inside the container won't stick, the value from `.env` is re-applied on every restart and silently wins. With this setup, the `.env` file is the single source of truth for the password; change it there, nowhere else.

Part B: Point your home network at it

Step 5: Point your home network at Pi-hole

This is the centerpiece. You want every device in the house to use Pi-hole as its DNS server without configuring each one. The cleanest way is to set it **once at your router**, because the router hands out DNS settings to every device via DHCP when they join the Wi-Fi.

1. Use the Pi's reserved LAN IP from Step 2 (e.g. `192.168.1.50`). That's the address every device will be told to use for DNS.
2. In the router's settings, find the **DNS server** field, usually under *DHCP*, *LAN*, or *Internet* settings, and set the **primary DNS** to the Pi's IP (e.g. `192.168.1.50`).
3. **Leave the secondary DNS blank if you can.** This is counterintuitive: a secondary DNS (like `8.8.8.8`) feels like a safety net, but devices freely use *either* server, so half your queries would skip Pi-hole and you'd see ads "randomly." A single DNS entry forces everything through Pi-hole.
4. Renew leases so devices pick up the change: reboot the router, or toggle Wi-Fi off/on on each device (or just wait, leases renew on their own).

> ℹ What if your ISP router won't let you change DNS?
>
> Some locked-down ISP gateways don't expose the DNS field. Two fallbacks: (a) set DNS **per-device** (each phone/laptop's Wi-Fi settings let you set a manual DNS, more tedious but works), or (b) let Pi-hole run your network's **DHCP** instead of the router (an option in Pi-hole's settings; you'd disable the router's DHCP first). The router approach above is by far the simplest when it's available.

> ⚠ Don't expose port 53 to the internet
>
> Whatever you do, never port-forward port 53 from your router to the Pi. An open, internet-facing DNS resolver gets abused for DNS-amplification attacks within hours. Pi-hole answering your *LAN* is exactly right; Pi-hole answering the *whole internet* is a problem. Keep it on your home network.

Part C: Add blocklists and verify

Step 6: Add good blocklists

A fresh Pi-hole v6 already subscribes to **StevenBlack's unified hosts list**, which is a strong baseline on its own. You can verify it under **Lists** in the UI. To add more coverage:

1. In the web UI, go to **Lists** (sometimes shown as "Adlists" or "Subscribed Lists").
2. Paste a list URL and click **Add**.
3. Crucially, go to **Tools → Update Gravity** (or run `docker exec pihole pihole -g`). Blocklists do *nothing* until "gravity", Pi-hole's compiled blocklist database, is rebuilt.

Two well-maintained additions that block aggressively without breaking sites. Each URL is in its own block below so it copies cleanly, paste one at a time into **Lists → Add**:

OISD Big, a popular all-in-one list that's deliberately conservative about breaking sites:

```
https://big.oisd.nl
```

HaGeZi Multi PRO, actively maintained and tiered ("PRO" is the balanced middle tier). The raw URL is long; copy the whole single line:

```
https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/pro.txt
```

> ⚠ Don't stack a dozen lists
>
> More lists is not more better. Two or three quality lists (StevenBlack + OISD *or* HaGeZi) outperform a sprawling pile of overlapping ones and cause far fewer "this website is broken" surprises. Every time you add or remove a list, run **Update Gravity** or nothing changes.

Step 7: Verify the whole thing

Run through these in order; each isolates a different layer:

```
# Query Pi-hole at the Pi's LAN IP (e.g. 192.168.1.50), the address it's
bound to.
# 1. On the Pi itself:
dig +short google.com @192.168.1.50          # returns IPs = resolver works
dig +short doubleclick.net @192.168.1.50     # returns 0.0.0.0 = blocking
works

# 2. From another device on the LAN (run this from your laptop at home):
dig +short doubleclick.net @192.168.1.50     # 0.0.0.0 = LAN clients can use
it
```

> **Why not `@127.0.0.1`?**
>
> You might reach for `dig @127.0.0.1` on the Pi, but here it returns "connection refused", and that's correct, not a fault. In this chapter Pi-hole is published on the **LAN IP only** (`${PIHOLE_IP}:53`), not on localhost, and Step 1 disabled the `systemd-resolved` stub that used to answer on `127.0.0.53`. So query the Pi by its LAN address instead. (In Chapter 4 you switch Pi-hole to listen on all interfaces, and `@127.0.0.1` starts working too.)

Then the real-world test: on a device connected to your home Wi-Fi, open a normally ad-heavy site or app, then watch the **Query Log** at your Pi's admin page (e.g. `http://192.168.1.50/admin`) from your laptop. You should see that device's LAN IP making queries, with ad domains marked blocked.

If a device's queries don't appear at all, it's still using its old DNS, renew its DHCP lease (toggle Wi-Fi off/on) so it picks up the router's new DNS setting.

Troubleshooting

Pi-hole container won't start / "port 53 already in use." The `systemd-resolved` stub is still bound. Re-run Step 1 and verify with `sudo ss -tulpn | grep ':53'`. You should see only Docker/Pi-hole, never `systemd-resolve`.

Some devices still show ads after the router DNS change. Either they haven't renewed their DHCP lease (toggle Wi-Fi off/on), or your router has a *secondary* DNS set that lets them bypass Pi-hole. Clear the secondary entry (Step 5.3).

A device hard-codes its own DNS and ignores the router. Some devices (notably certain Android phones, Chromecasts, and anything using DNS-over-HTTPS to `8.8.8.8`) bypass your router's DNS entirely. You can't fix those from the router; either change DNS on the device itself or block their hard-coded resolvers, an advanced topic, but worth knowing it's *them*, not a broken Pi-hole.

A specific site is broken. A blocklist is over-matching. Find the blocked domain in the **Query Log**, click it, and **Allow** it. Then the site works while everything else stays blocked.

You lost the admin password. It's in your `.env` file, `cat ~/pihole/.env` and read `PIHOLE_PASSWORD`. (This is the upside of keeping it in `.env`: it's recoverable from a file only you can read, not lost in a container.)

The Pi itself can't resolve names after Step 1. Check that `/etc/resolv.conf` is the symlink from Step 1.2 and that your upstream (`FTLCONF_dns_upstreams`) is a real public resolver, not the router, if the router points back at Pi-hole (that's a resolution loop).

Recap

- **Free port 53** by disabling the `systemd-resolved` stub listener.
- **Run Pi-hole** in Docker Compose next to Audiobookshelf.
- **Point your home network at it** by setting the Pi as the router's DNS server (single entry, no secondary), one setting covers every device.

- **Add one or two quality blocklists** and run **Update Gravity**.

Your homelab now does two jobs on your home network. In Chapter 4 we stop typing IP addresses and ports to reach these services, a small reverse proxy plus local DNS gives Pi-hole and Audiobookshelf clean names like `https://pihole.home` and `https://abs.home` that work from anywhere on your tailnet.